



PROYECTO INDIVIDUAL: APLICACIÓN DE RESERVAS DE SPA

ASIGNATURA: PROYECTO INTERMODULAR 2



28 DE MAYO DE 2026
ISLENA ISABEL POLO FRANCO
2ND SEMI DESARROLLO DE APLICACIONES WEB

Contenido

1.	¡Error! Marcador no definido.
Resumen del proyecto	3
2. Objetivo principal	3
3. Tecnologías utilizadas.....	3
4. Requisitos del sistema	4
4.1. Hardware necesario	4
4.2. Software necesario.....	4
4.3. Versiones recomendadas	5
5. Arquitectura de la aplicación	5
5.1. Estructura general de carpetas y ficheros	5
6. Módulos y funcionalidades.....	6
6.1. Módulo de gestión de spas	6
6.2. Módulo de gestión de servicios	7
Gestiona los tratamientos y servicios disponibles en cada spa.....	7
6.3. Módulo de reservas.....	7
6.4. Módulo de empleados.....	8
6.5. Módulo de correos y notificaciones.....	8
7. Interfaz de usuario	10
7.1. Capturas de pantalla de las principales secciones	10
Sección de selección de spa:.....	10
7.2. Explicación de la interacción con el usuario	17
8. Instalación y despliegue.....	18
8.1. Clonación del repositorio o descarga del proyecto.....	18
8.2 Configuración del entorno	18
8.3. Cómo ejecutar la aplicación en local	19
8.4 Cómo desplegar en servidor/hosting	22
8.5. Problemas comunes y soluciones	23
9. Control de versiones y GitHub	24
9.1. URL del repositorio	24
9.2. Estructura de ramas	24
9.3. Convenciones de commits	26
9.4. Cómo revisar el historial de versiones.....	26
9.5 Capturas de hitos importantes del proyecto.....	28

10. Pruebas y validación.....	31
10.1. Casos de prueba principales	31
10.2. Pruebas de funcionalidad e integración	33
10.3. Pruebas de accesibilidad.....	33
11. Seguridad y buenas prácticas	34
11.1 Medidas de seguridad implementadas	34
11.2 Buenas prácticas adoptadas	35
12. Mantenimiento y futuras mejoras	35
12.1. Cómo actualizar y mantener el proyecto	35
12.2. Funcionalidades pendientes o ideas futuras.....	36
13. Conclusión.....	37
Valoración del proyecto	37
Aprendizajes obtenidos.....	38
14. Anexo.....	39
14.1. Código relevante comentado	39
14.2. Diagramas adicionales:.....	47

1. Resumen del proyecto

Este proyecto consiste en el desarrollo de una aplicación web de gestión y reservas para centros de spa y bienestar. La plataforma permite a los clientes consultar servicios, realizar reservas online y gestionar sus citas de forma sencilla e intuitiva. Además, incluye un sistema de administración dividido en dos paneles diferenciados según el rol del usuario.

Por un lado, existe un panel de WebMaster, encargado de la supervisión global de la plataforma. Este panel permite gestionar todos los spas registrados en el sistema, controlar incidencias, administrar información general y actuar en caso de problemas técnicos o de gestión dentro de cualquier empresa registrada.

Por otro lado, cada empresa dispone de su propio panel de administración privado, desde el cual puede gestionar sus empleados, servicios, categorías, horarios y reservas de forma independiente. De esta manera, cada spa mantiene el control de su propio negocio mientras la plataforma centraliza la administración global.

La aplicación se encuentra desplegada y puede consultarse desde la siguiente URL:

<https://easyspa.onrender.com/>

2. Objetivo principal

El objetivo principal del proyecto es digitalizar y optimizar la gestión de reservas en centros de spa, facilitando tanto la experiencia de los clientes como la administración interna de las empresas. A través de esta plataforma, se busca reducir la gestión manual de citas, mejorar la organización de los empleados y ofrecer una solución accesible desde cualquier dispositivo con conexión a internet.

Asimismo, el sistema pretende proporcionar a los negocios una herramienta moderna que les permita administrar sus servicios, controlar la disponibilidad de sus trabajadores y gestionar las reservas de manera eficiente y automatizada.

3. Tecnologías utilizadas

Para el desarrollo del proyecto se ha utilizado un conjunto de tecnologías modernas orientadas al desarrollo web full stack.

En el **back-end** se ha utilizado Laravel, un framework de PHP que permite desarrollar aplicaciones robustas siguiendo el patrón MVC. Laravel se ha empleado para la creación de la API REST, la autenticación de usuarios, la validación de datos, la gestión de reservas y la comunicación con la base de datos.

En el **front-end** se ha utilizado React junto con Vite. React ha permitido construir una interfaz moderna basada en componentes reutilizables, mientras que Vite se ha utilizado

como herramienta de desarrollo y compilación para mejorar el rendimiento y la velocidad de carga durante el desarrollo.

Como sistema de almacenamiento de datos se ha empleado MySQL, una base de datos relacional encargada de almacenar la información de usuarios, reservas, servicios, empleados y categorías.

Para el control de versiones y la gestión del código fuente se ha utilizado Git, permitiendo organizar el desarrollo mediante ramas y mantener un seguimiento de los cambios realizados durante el proyecto.

Finalmente, para el entorno de despliegue y producción se ha utilizado Docker, facilitando la contenerización de la aplicación y asegurando un entorno estable y reproducible para su ejecución.

4. Requisitos del sistema

Para el correcto funcionamiento y desarrollo de la aplicación se requiere un entorno capaz de ejecutar aplicaciones web modernas tanto en el servidor como en el cliente. El sistema ha sido desarrollado y probado en un entorno local utilizando herramientas compatibles con el ecosistema de desarrollo de PHP y JavaScript.

4.1. Hardware necesario

No se requieren requisitos hardware especialmente elevados. Se recomienda disponer de:

- Procesador de 64 bits.
- Mínimo 8 GB de memoria RAM.
- Al menos 10 GB de espacio libre en disco.
- Conexión a internet para instalación de dependencias y acceso a servicios externos.

4.2. Software necesario

Para ejecutar y desarrollar la aplicación se recomienda disponer del siguiente software:

- Sistema operativo Windows 11 o cualquier sistema operativo compatible con PHP y Node.js.
- PHP para la ejecución del backend desarrollado en Laravel.
- Composer para la gestión de dependencias PHP.
- Node.js y npm para la instalación y gestión de dependencias del frontend.
- Laravel como framework backend principal.
- React y Vite para el desarrollo de la interfaz de usuario.
- MySQL como sistema gestor de base de datos relacional.
- Git para el control de versiones del proyecto.
- Visual Studio Code o cualquier editor de código compatible con PHP y TypeScript.

4.3. Versiones recomendadas

Las versiones recomendadas para garantizar la compatibilidad del proyecto son las siguientes:

- Windows 11.
- PHP 8.2 o superior.
- Laravel 11.
- Node.js 20 o superior.
- MySQL 8.
- Git 2.40 o superior.

5. Arquitectura de la aplicación

5.1. Estructura general de carpetas y ficheros

La aplicación sigue una estructura organizada basada en Laravel como framework backend y React con Vite para el frontend. Todo el proyecto se encuentra integrado en una única aplicación, lo que facilita la comunicación entre cliente y servidor y simplifica el despliegue en producción.

A continuación, se describen las carpetas y archivos más importantes del proyecto:

Carpeta app: Contiene toda la lógica principal del backend desarrollada en Laravel. En esta carpeta se encuentran los controladores, modelos, servicios y demás clases encargadas de gestionar las funcionalidades de la aplicación y la comunicación con la base de datos.

Carpeta Bootstrap: Incluye archivos necesarios para el arranque de Laravel y la carga automática de dependencias.

Carpeta config: Almacena los archivos de configuración del sistema, como la conexión con la base de datos, correo electrónico, autenticación y otras configuraciones generales del proyecto.

Carpeta database: Contiene las migraciones, seeders y factories utilizados para crear y poblar la base de datos MySQL.

Carpeta public: Es la carpeta pública accesible desde el navegador. Aquí se almacenan archivos generados durante la compilación del frontend, imágenes públicas y el punto de entrada principal de la aplicación.

Carpeta resources: Es una de las carpetas más importantes del proyecto, ya que contiene tanto los recursos visuales como toda la aplicación frontend desarrollada con React y TypeScript:

Dentro de esta carpeta destacan:

- components/: componentes reutilizables de React utilizados en diferentes vistas.
- context/: gestión del estado global mediante Context API.
- hooks/: contiene hooks personalizados de React que permiten reutilizar lógica entre diferentes componentes. En ellos se centralizan operaciones como peticiones a la API, gestión de formularios, estados de carga, errores, filtros y operaciones CRUD. Para mantener una estructura clara, se divide en cuatro carpetas según el área de la aplicación: WebMaster, Admin, Client y Public.
- pages/: vistas principales de la aplicación organizadas por funcionalidades y paneles.
- types/: interfaces y tipos TypeScript utilizados para mantener un código más seguro y organizado.
- Carpeta resources/views: Contiene las vistas Blade de Laravel utilizadas principalmente para la integración inicial con React y algunos elementos del backend.

Carpeta **routes**: Incluye la definición de rutas del backend, tanto web como API.

Carpeta **storage**: Almacena archivos generados dinámicamente, como imágenes subidas por usuarios, logs del sistema y archivos temporales.

Carpeta **tests**: Contiene pruebas automáticas del proyecto.

Carpeta **vendor**: Incluye todas las dependencias PHP instaladas mediante Composer.

Archivos importantes del proyecto

.env: archivo de configuración con variables de entorno sensibles como credenciales de base de datos o correo.

composer.json: dependencias y configuración del backend Laravel.

package.json: dependencias y scripts del frontend React/Vite.

dockerignore: configuración utilizada para excluir archivos en el despliegue con Docker.

editorconfig: configuración de formato y estilo del código fuente.

6. Módulos y funcionalidades

6.1. Módulo de gestión de spas

Permite administrar toda la información relacionada con los spas registrados en la plataforma.

Funcionalidades:

- Crear spas

- Editar información de spas
- Eliminar spas
- Gestión de imágenes
- Configuración de horarios y datos de contacto
- Asociación de empleados y servicios
- Visualización de disponibilidad mediante calendarios interactivos

Dependencias:

- Laravel
- MySQL
- React
- Hooks personalizados
- Context API
- FullCalendar

6.2. Módulo de gestión de servicios

Gestiona los tratamientos y servicios disponibles en cada spa.

Funcionalidades

- Crear servicios
- Editar servicios
- Eliminar servicios
- Gestión de precios y duración
- Configuración de capacidad máxima
- Asociación de servicios a un spa
- Visualización de horarios y disponibilidad en calendario

Dependencias:

- Laravel API REST
- React
- Axios
- MySQL
- Hooks personalizados
- Context API
- FullCalendar

6.3. Módulo de reservas

Es uno de los módulos principales de la aplicación y permite realizar y gestionar reservas.

Funcionalidades

- Creación de reservas
- Consulta de disponibilidad
- Validación de horarios
- Cancelación de reservas
- Gestión de estados de reserva
- Asociación de clientes, servicios y empleados
- Visualización de reservas en calendarios interactivos

Dependencias

- React
- Laravel
- MySQL
- Hooks personalizados
- Context API
- FullCalendar

6.4. Módulo de empleados

Gestiona los empleados asociados a cada spa.

Funcionalidades

- Alta de empleados
- Edición de información
- Eliminación de empleados
- Asignación de horarios
- Asociación con servicios

Dependencias

- Laravel
- React
- MySQL

6.5. Módulo de correos y notificaciones

Gestiona el envío automático de correos electrónicos relacionados con reservas y recordatorios.

Funcionalidades

- Envío de confirmaciones de reserva
- Generación de PDFs de confirmación
- Recordatorios automáticos
- Sistema de colas para procesamiento en segundo plano

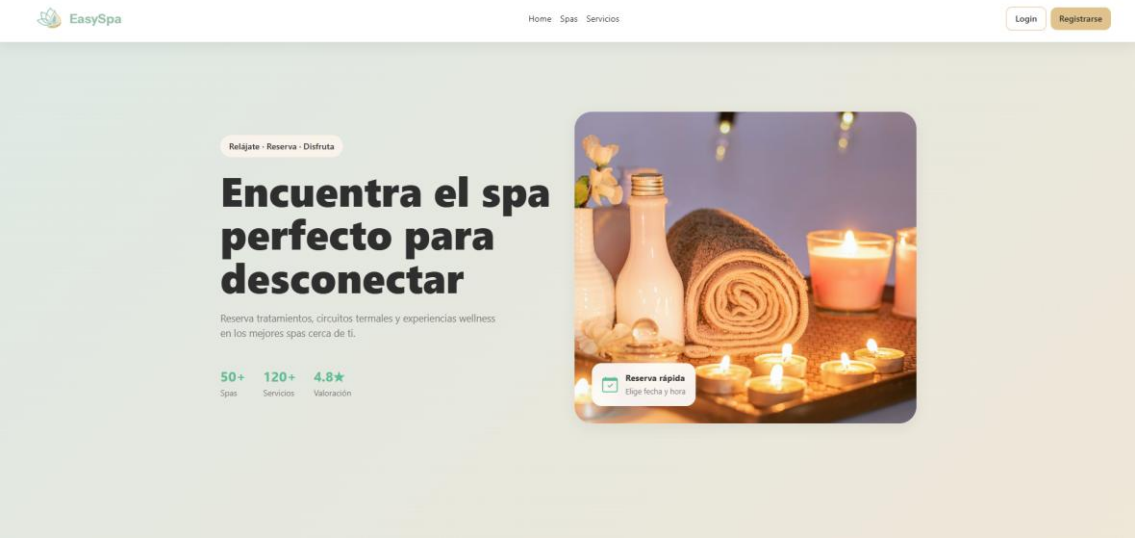
Dependencias

- Laravel Mail
- Queue Workers
- DomPDF
- SMTP

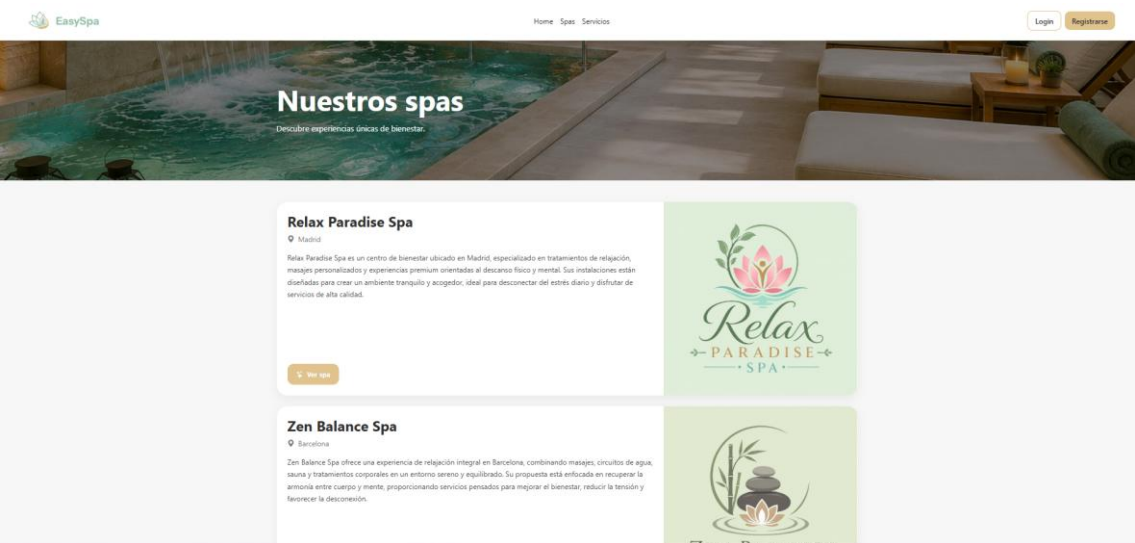
7. Interfaz de usuario

7.1. Capturas de pantalla de las principales secciones

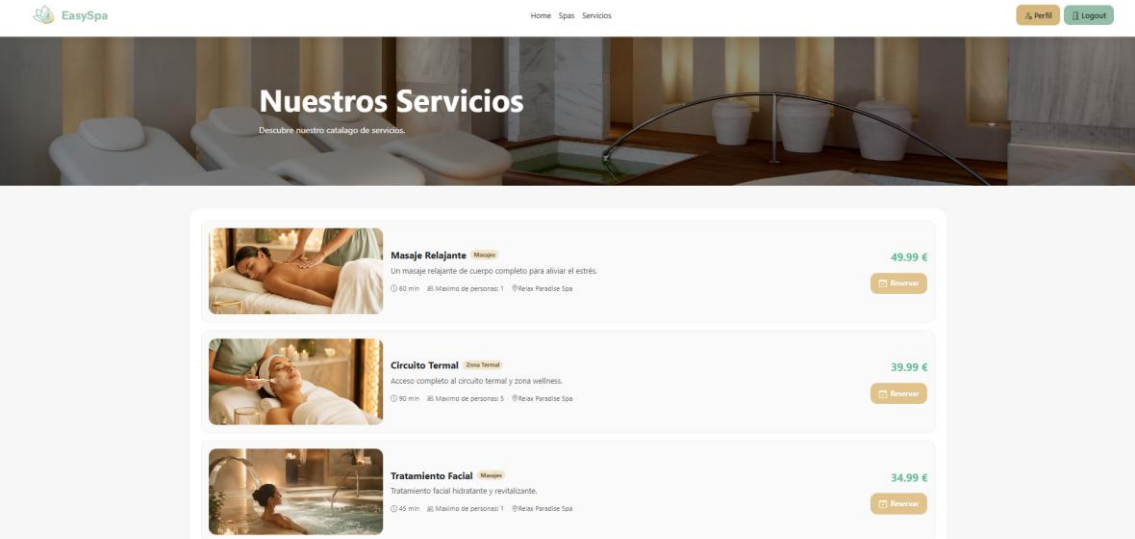
Pagina principal:



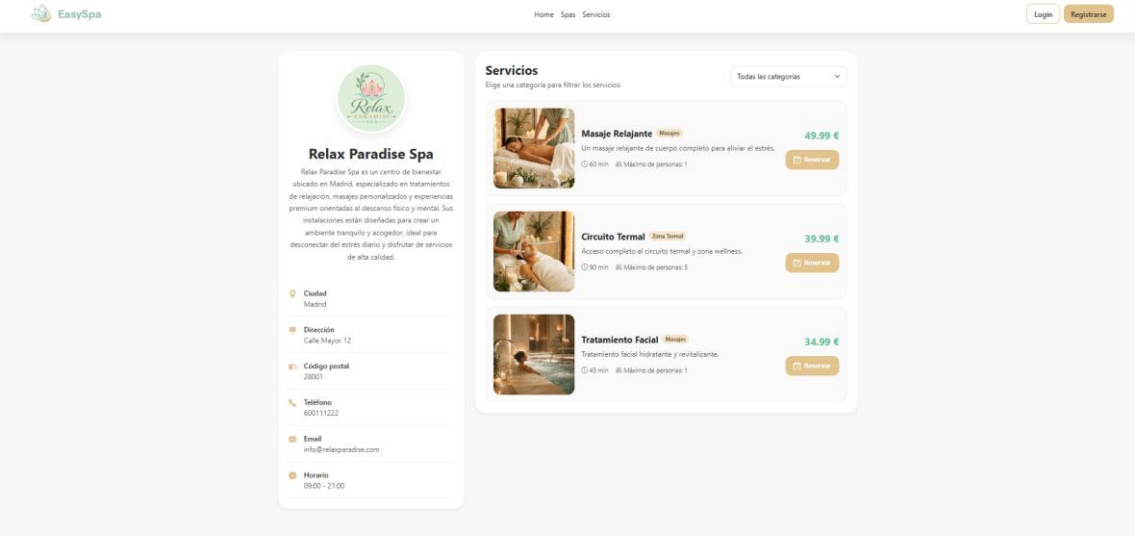
Sección de selección de spa:



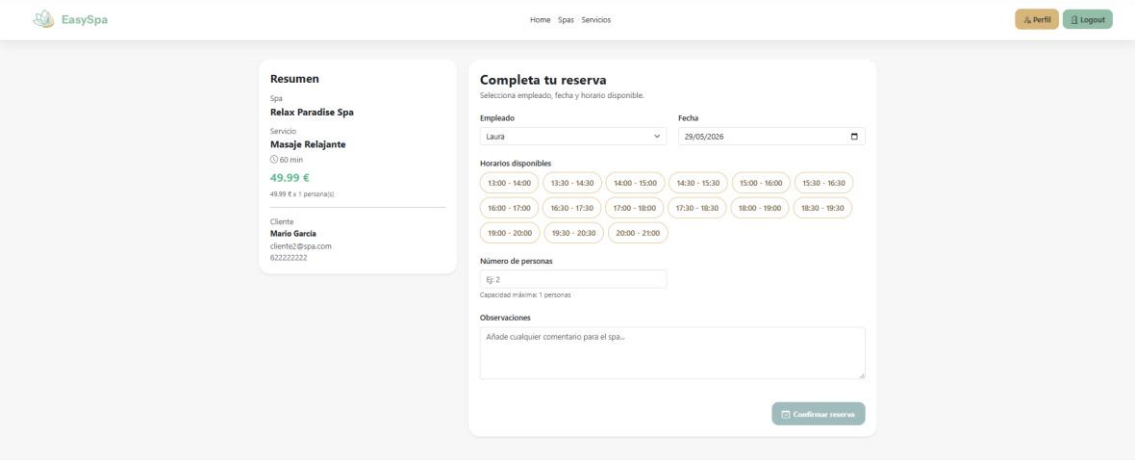
Listado global de todos los servicios:



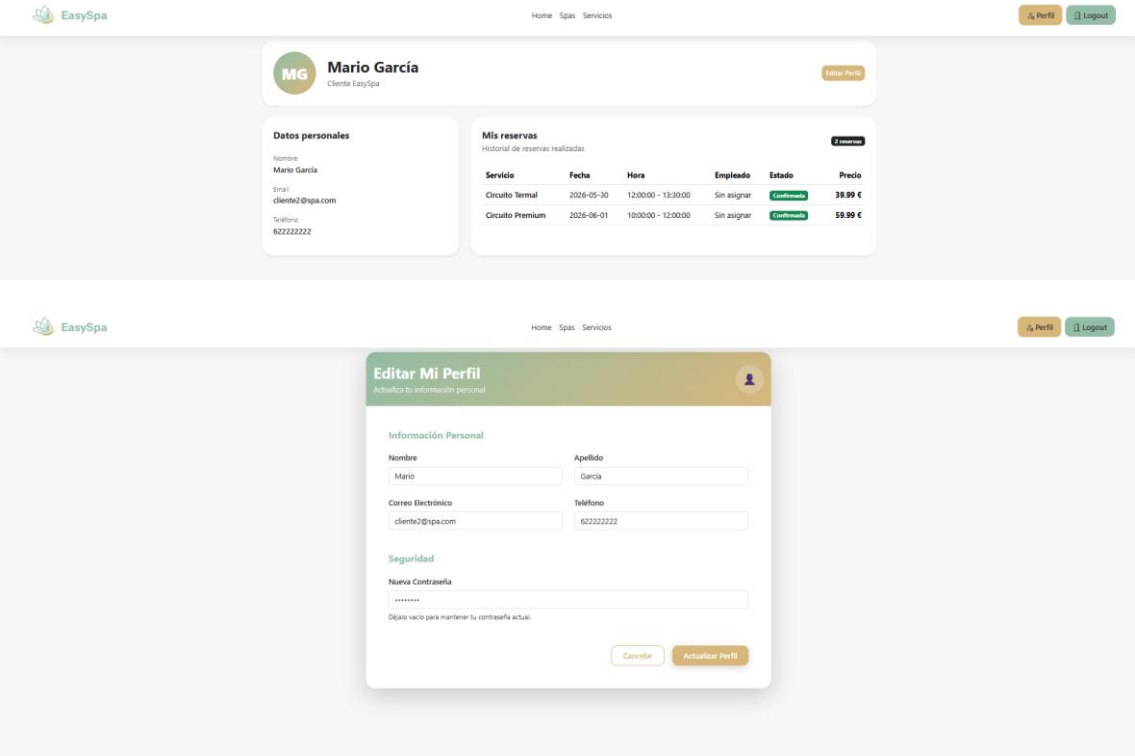
Sección de cada spa en el que se muestran sus servicios y da la opción de reservar:



Sección de reserva:

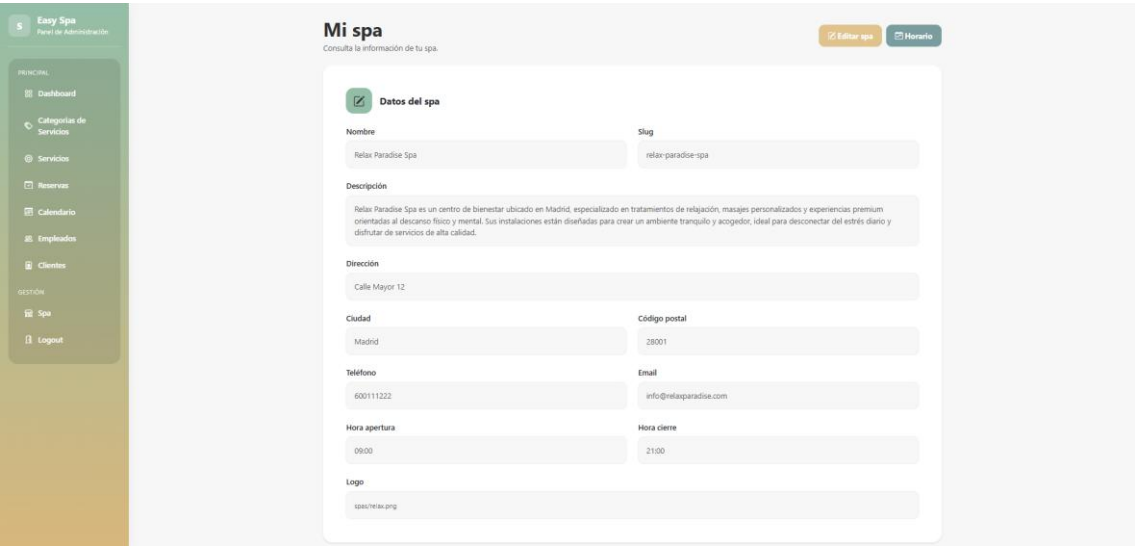


Perfil del cliente:



Panel de administración de empresa:

Datos del spa y horario:



5

Easy Spa

Panel de Administración

PRINCIPAL

Dashboard

Categorías de Servicios

Servicios

Reservas

Calendario

Empleados

Clientes

GESTIÓN

Spa

Login

admin1@spa.com

Admin

A

Horario del spa

Día	Abierto	Inicio	Fin
Domingo	☐	09:00	17:00
Lunes	☒	09:00	21:00
Martes	☒	09:00	21:00
Miércoles	☒	09:00	21:00
Jueves	☒	09:00	21:00
Viernes	☒	09:00	21:00
Sábado	☐	09:00	17:00

Guardar horario

Reservas:

5

Easy Spa

Panel de Administración

PRINCIPAL

Dashboard

Categorías de Servicios

Servicios

Reservas

Calendario

Empleados

Clientes

GESTIÓN

Spa

Login

admin1@spa.com

Admin

A

Reservas

Gestiona las reservas de tu spa.

Nueva reserva

Cliente	Servicio	Empleado	Fecha	Hora	Estado	Acciones
Lucía Fernández	Masaje Relajante	Carlos Martínez	2026-05-29	10:00 - 11:00	confirmed	Editar Ver Eliminar
Mario García	Circuito Termal	Sin asignar	2026-05-30	12:00 - 13:30	confirmed	Editar Ver Eliminar
Sara López	Tratamiento Facial	Laura Gómez	2026-05-31	16:00 - 16:45	pending	Editar Ver Eliminar

5

Easy Spa

Panel de Administración

PRINCIPAL

Dashboard

Categorías de Servicios

Servicios

Reservas

Calendario

Empleados

Clientes

GESTIÓN

Spa

Login

Detalle reserva

Información completa de la reserva

Volver

Resumen

Lucía Fernández reservó Masaje Relajante para el día 2026-05-29 de 10:00:00 a 11:00:00.

Datos de la reserva

Empleado asignado

Acciones

Servicios:

5

Easy Spa

Panel de Administración

PRINCIPAL

Dashboard

Categorías de Servicios

Servicios

Reservas

Calendario

Empleados

Clientes

GESTIÓN

Spa

Logout

admin@spa.com

Admin

A

Servicios

Gestiona los servicios de tu spa.

Nuevo servicio

Nombre	Categoría	Duración	Precio	Capacidad	Activo	Acciones
Círculo Termal	Zona Termal	90 min	39.99 €	5	Activo	Ver Editar
Masaje Relajante	Masajes	60 min	49.99 €	1	Activo	Ver Editar
Tratamiento Facial	Masajes	45 min	34.99 €	1	Activo	Ver Editar

Detalles del empleado y sus horarios:

5

Easy Spa

Panel de Administración

PRINCIPAL

Dashboard

Categorías de Servicios

Servicios

Reservas

Calendario

Empleados

Clientes

GESTIÓN

Spa

Logout

admin@spa.com

Admin

A

Empleados

Gestiona los empleados de tu spa.

Nuevo empleado

Nombre	Email	Teléfono	Activo	Acciones
Carlos Martínez	carlos@spa1.com	600000001	Activo	Ver Editar
Laura Gómez	laura@spa1.com	600000002	Activo	Ver Editar
David Ruiz	david@spa1.com	600000003	Activo	Ver Editar

5

Easy Spa

Panel de Administración

PRINCIPAL

Dashboard

Categorías de Servicios

Servicios

Reservas

Calendario

Empleados

Clientes

GESTIÓN

Spa

Logout

admin@spa.com

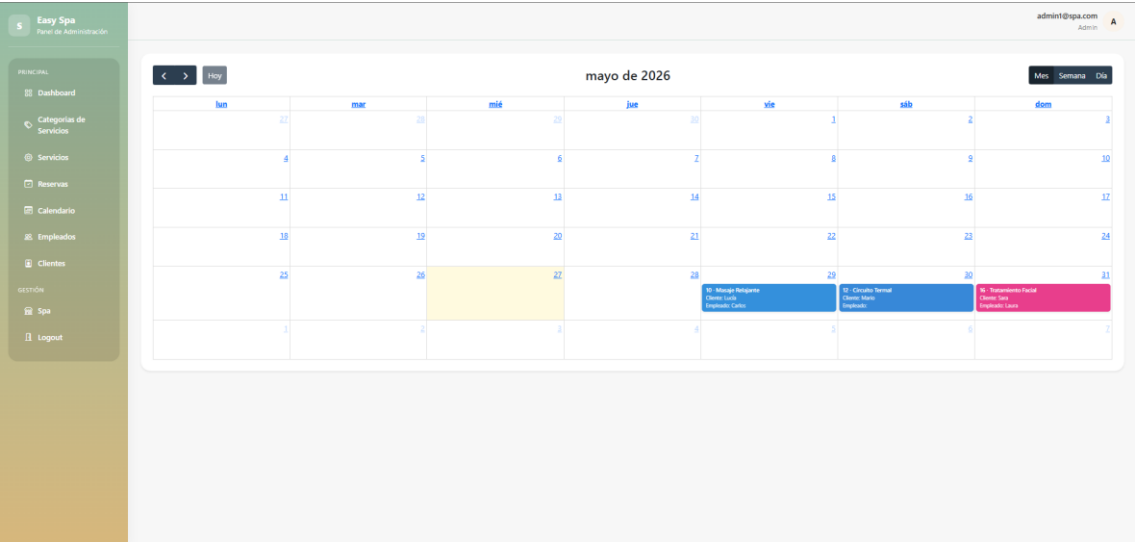
Admin

A

Horario mensual

Fecha	Día	Trabaja	Inicio	Fin
2026-05-01	Vie	☒	09:00 ⌚	17:00 ⌚
2026-05-02	Sáb	☒	09:00 ⌚	17:00 ⌚
2026-05-03	Dom	☒	09:00 ⌚	17:00 ⌚
2026-05-04	Lun	☒	09:00 ⌚	17:00 ⌚
2026-05-05	Mar	☒	09:00 ⌚	17:00 ⌚
2026-05-06	Mié	☒	09:00 ⌚	17:00 ⌚
2026-05-07	Jue	☒	09:00 ⌚	17:00 ⌚
2026-05-08	Vie	☒	09:00 ⌚	17:00 ⌚
2026-05-09	Sáb	☒	09:00 ⌚	17:00 ⌚
2026-05-10	Dom	☒	09:00 ⌚	17:00 ⌚
2026-05-11	Lun	☒	09:00 ⌚	17:00 ⌚
2026-05-12	Mar	☒	09:00 ⌚	17:00 ⌚
2026-05-13	Mié	☒	09:00 ⌚	17:00 ⌚
2026-05-14	Jue	☒	09:00 ⌚	17:00 ⌚
2026-05-15	Vie	☒	09:00 ⌚	17:00 ⌚
2026-05-16	Sáb	☒	09:00 ⌚	17:00 ⌚
2026-05-17	Dom	☒	09:00 ⌚	17:00 ⌚
2026-05-18	Lun	☒	09:00 ⌚	17:00 ⌚
2026-05-19	Mar	☒	09:00 ⌚	17:00 ⌚

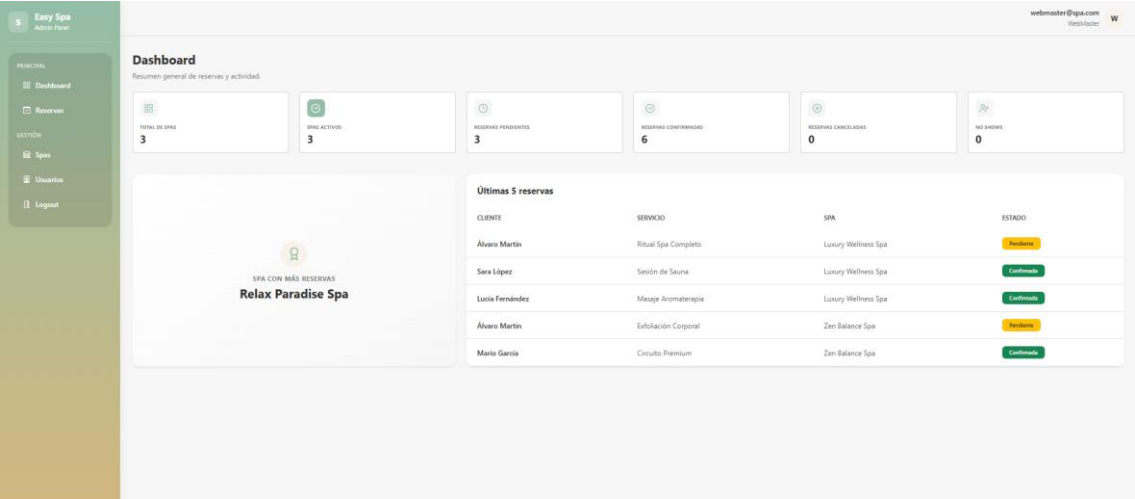
Calendario:



Panel del webmaster:

El panel del webmaster mantiene una estructura similar al panel de administración, por lo que no se han incluido capturas repetidas de vistas con un funcionamiento equivalente. La diferencia principal entre ambos roles es que el webmaster tiene acceso global a todos los spas registrados, mientras que el administrador solo gestiona el spa asociado a su cuenta. Por ello, en este apartado se muestran únicamente las capturas más representativas del rol webmaster, centradas en la gestión y supervisión general de la plataforma.

Dashboard:



Listado de Spas:

S

Easy Spa

Admin Panel

PRINCIPAL

Dashboard

Reservas

GESTIÓN

Spas

Usuarios

Logout

webmaster@spa.com

WebMaster

W

Spas

Gestiona todos los spas del sistema.

Crear spa

Listado de spas

Nombre	Ciudad	Teléfono	Email	Horario	Estado	Acciones
Relax Paradise Spa relax-paradise-spa	Madrid	600111222	info@relaxparadise.com	09:00:00 - 21:00:00	Activo	Ver Editar Eliminar
Zen Balance Spa zen-balance-spa	Barcelona	611333444	contacto@zenbalance.com	10:00:00 - 22:00:00	Activo	Ver Editar Eliminar
Luxury Wellness Spa luxury-wellness-spa	Valencia	622555666	hello@luxurywellness.com	08:30:00 - 20:30:00	Activo	Ver Editar Eliminar

Usuarios:

S

Easy Spa

Admin Panel

PRINCIPAL

Dashboard

Reservas

GESTIÓN

Spas

Usuarios

Logout

webmaster@spa.com

WebMaster

W

Usuarios

Gestión de usuarios del sistema

Crear usuario

Filtrar por rol

Todos los roles

Filtrar por spa

Todos los spas

Limpiar filtros

ID	Email	Rol	Acciones
4	admin3@spa.com	Admin	Ver Editar Eliminar
5	cliente1@spa.com	Client	Ver Editar Eliminar
6	cliente2@spa.com	Client	Ver Editar Eliminar
7	cliente3@spa.com	Client	Ver Editar Eliminar
8	cliente4@spa.com	Client	Ver Editar Eliminar
1	webmaster@spa.com	WebMaster	Ver Editar Eliminar
2	admin1@spa.com	Admin	Ver Editar Eliminar
3	admin2@spa.com	Admin	Ver Editar Eliminar

Reservas Globales:

S

Easy Spa

Admin Panel

PRINCIPAL

Dashboard

Reservas

GESTIÓN

Spas

Usuarios

Logout

webmaster@spa.com

WebMaster

W

Reservas globales

Consulta todas las reservas registradas en todos los spas.

Volver

Buscar

Spa

Estado

Fecha

Cliente, servicio o spa...

Todos los spas

Todos

dd/mm/aaaa

Limpiar filtros

Listado de reservas

9 reservas

Spa	Cliente	Servicio	Fecha	Hora	Estado	Precio	Acciones
Relax Paradise Spa	Lucia Fernández cliente1@spa.com	Masaje Relajante	2026-05-29	10:00:00 - 11:00:00	Confirmada	49.99 €	Ver Eliminar
Relax Paradise Spa	Mario Garcia cliente2@spa.com	Circuito Termal	2026-05-30	12:00:00 - 13:30:00	Confirmada	39.99 €	Ver Eliminar
Relax Paradise Spa	Sara López cliente3@spa.com	Tratamiento Facial	2026-05-31	16:00:00 - 16:45:00	Pendiente	34.99 €	Ver Eliminar
Zen Balance Spa	Lucia Fernández cliente1@spa.com	Masaje en Pareja	2026-05-29	11:00:00 - 12:00:00	Confirmada	54.99 €	Ver Eliminar
Zen Balance Spa	Mario Garcia cliente2@spa.com	Circuito Premium	2026-06-01	10:00:00 - 12:00:00	Confirmada	59.99 €	Ver Eliminar
Zen Balance Spa	Álvaro Martín cliente4@spa.com	Exfoliación Corporal	2026-06-02	17:00:00 - 17:50:00	Pendiente	44.99 €	Ver Eliminar

Lucia Fernández

Página 16 de 47

7.2. Explicación de la interacción con el usuario

La aplicación dispone de tres tipos principales de usuarios: cliente, empresa de spa y webmaster. Cada uno cuenta con diferentes permisos y funcionalidades dentro del sistema.

Cliente

El cliente es el usuario encargado de realizar reservas dentro de la plataforma. Puede acceder libremente a la página principal y consultar los spas disponibles, así como los servicios ofrecidos por cada uno de ellos. Una vez registrado o autenticado en la aplicación, el cliente puede:

- Realizar reservas de servicios.
- Consultar la disponibilidad y horarios.
- Acceder a su perfil personal.
- Modificar sus datos personales.
- Consultar el historial de reservas realizadas.
- Recibir correos de confirmación y recordatorios automáticos.

La interacción del cliente está orientada a facilitar el proceso de búsqueda y reserva de servicios de spa de forma sencilla e intuitiva.

Empresa de spa

La empresa de spa dispone de un panel de gestión propio desde el cual administra toda la información relacionada con su negocio dentro de la plataforma. Entre sus funcionalidades principales se encuentran:

- Crear y editar spas.
- Gestionar servicios y tratamientos.
- Gestionar empleados.
- Configurar horarios y disponibilidad.
- Visualizar reservas mediante calendarios interactivos.
- Crear reservas manualmente para clientes.
- Gestionar imágenes e información del spa.

Este tipo de usuario actúa como administrador de su propio spa, teniendo control únicamente sobre los recursos asociados a su empresa.

Webmaster

El webmaster es el administrador global de la plataforma y dispone de acceso completo al sistema. Sus principales funcionalidades son:

- Crear, editar y eliminar usuarios.
- Crear, editar y eliminar spas.
- Supervisar todas las reservas del sistema.
- Gestionar incidencias.
- Administrar el contenido global de la plataforma.
- Controlar el correcto funcionamiento de la aplicación.

Este rol tiene permisos superiores al resto de usuarios y permite mantener el control general de toda la plataforma EasySpa.

8. Instalación y despliegue

8.1. Clonación del repositorio o descarga del proyecto

Para obtener el proyecto en un entorno local, se puede clonar el repositorio desde GitHub mediante Git o descargarlo directamente en formato comprimido.

En caso de utilizar Git, el comando sería:

```
git clone https://github.com/islena17/EasySpa
```

Una vez clonado el repositorio, se accede a la carpeta principal del proyecto:

```
cd easy_spa
```

También es posible descargar el proyecto manualmente como archivo .zip, descomprimirlo y abrirlo desde el editor de código utilizado, como Visual Studio Code (el que yo personalmente he usado para realizar el proyecto). Tras obtener los archivos del proyecto, se debe continuar con la instalación de dependencias, configuración del entorno y preparación de la base de datos.

8.2 Configuración del entorno

Para ejecutar correctamente el proyecto, es necesario configurar las variables de entorno del archivo .env. Este archivo contiene los datos necesarios para conectar la aplicación con los servicios externos y recursos principales del sistema.

En primer lugar, se configura la conexión con la base de datos MySQL, indicando el nombre de la base de datos, el usuario, la contraseña, el host y el puerto correspondiente.

También se definen las variables relacionadas con el servidor de la aplicación, como la URL del backend y la URL del frontend, necesarias para que Laravel y React puedan comunicarse correctamente.

Además, se configuran otros servicios utilizados por la aplicación, como el sistema de envío de correos mediante SMTP, las colas de trabajo para procesos en segundo plano y las credenciales necesarias para servicios externos, como Google Calendar.

Esta configuración permite adaptar el proyecto tanto a un entorno local de desarrollo como a un entorno de producción, modificando únicamente las variables del archivo `.env`.

8.3. Cómo ejecutar la aplicación en local

Para ejecutar el proyecto en un entorno local, es necesario disponer previamente de PHP, Composer, Node.js, MySQL y Git instalados en el sistema.

Una vez clonado o descargado el proyecto, se deben seguir los siguientes pasos:

Instalar dependencias del backend

Desde la carpeta principal del proyecto, ejecutar:

```
composer install
```

Este comando instalará todas las dependencias necesarias de Laravel.

Instalar dependencias del frontend

```
npm install
```

Con ello se instalarán todas las librerías utilizadas en React, Vite y el resto de dependencias del frontend.

Configurar el archivo `.env`

Se debe crear el archivo `.env` a partir del archivo `.env.example`:

```
cp .env.example .env
```

Posteriormente, se configuran las variables de entorno necesarias, como la conexión a la base de datos, correo electrónico y APIs externas.

Generar la clave de Laravel

```
php artisan key:generate
```

Configurar la base de datos

Crear una base de datos MySQL y ejecutar las migraciones:

```
php artisan migrate
```

En caso de querer cargar datos de prueba:

```
php artisan db:seed
```

Crear el enlace simbólico de almacenamiento

```
php artisan storage:link
```

Esto permitirá acceder correctamente a las imágenes almacenadas por la aplicación.

Configurar la URL base de Axios

Si el proyecto se descarga desde GitHub, es necesario revisar el archivo axios.ts, ya que la aplicación desplegada en Render utiliza como URL base la dirección del despliegue. Para ejecutar el proyecto en local, dicha URL debe modificarse por la dirección del servidor local de Laravel, por ejemplo:

```
http://localhost:8000
```

o bien:

```
http://127.0.0.1:8000
```

De esta forma, las peticiones realizadas desde el frontend se dirigirán correctamente al backend ejecutado en local.

Ejecutar el backend de Laravel

```
php artisan serve
```

Por defecto, el servidor estará disponible en:

```
http://127.0.0.1:8000
```

Ejecutar el frontend de React

En otra terminal:

```
npm run dev
```

El frontend quedará disponible normalmente en:

```
http://localhost:5173
```

Configuración del mailer y SMTP

Para que la aplicación pueda enviar correos electrónicos, como confirmaciones de reserva o recordatorios automáticos, es necesario configurar correctamente el servicio de correo en el archivo .env.

En este proyecto se ha utilizado **Gmail** como proveedor SMTP. Para ello, fue necesario crear una **contraseña de aplicación** desde la cuenta de Google, ya que Gmail no permite utilizar directamente la contraseña normal de la cuenta para este tipo de conexiones externas.

La configuración utilizada se basa en el servidor SMTP de Gmail:

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=correo@gmail.com
MAIL_PASSWORD=contraseña_de_aplicación
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=correo@gmail.com
MAIL_FROM_NAME="EasySpa"
```

Gracias a esta configuración, Laravel puede enviar correos *automáticos* desde la aplicación utilizando el servicio SMTP de Gmail.

Ejecutar el sistema de colas

Para el envío de correos y recordatorios automáticos es necesario iniciar el worker de Laravel:

```
php artisan queue:work
```

Configuración de la API de Google Calendar

Para la integración con Google Calendar fue necesario crear una cuenta en Google Cloud y habilitar el uso de la API de Google Calendar.

Desde la consola de Google Cloud se configuró el proyecto correspondiente y se generaron las credenciales necesarias para poder utilizar la API desde la aplicación. Estas credenciales permiten que el sistema pueda conectarse con Google Calendar y gestionar eventos relacionados con las reservas.

La clave generada por Google es información sensible, por lo que no debe subirse al repositorio público del proyecto ni incluirse directamente en la memoria. Por motivos de seguridad, esta clave únicamente se incluye en el archivo `.env` entregado dentro del archivo ZIP del proyecto.

Un ejemplo de configuración sería:

```
GOOGLE_CALENDAR_API_KEY=clave_privada
GOOGLE_CLIENT_ID=client_id
GOOGLE_CLIENT_SECRET=client_secret
GOOGLE_REDIRECT_URI=http://localhost:8000/google/callback
```

De esta forma, la aplicación puede utilizar Google Calendar en entorno local siempre que el archivo `.env` contenga las credenciales correctas.

8.4 Cómo desplegar en servidor/hosting

Para el despliegue de la aplicación se ha utilizado Render como plataforma de hosting, ya que es una de las pocas opciones gratuitas que permite alojar proyectos desarrollados con Laravel y React. Para poder desplegar correctamente la aplicación en esta plataforma fue necesario crear un archivo Dockerfile, encargado de preparar el entorno de producción e iniciar el proyecto.

En primer lugar, es necesario crear una cuenta en Render, preferiblemente vinculándola con la cuenta de GitHub. De esta forma, Render puede acceder directamente al repositorio donde se encuentra alojado el proyecto.

Una vez creada la cuenta, se debe crear un nuevo servicio web para alojar la aplicación. Durante este proceso, se selecciona el repositorio del proyecto y se indica que el despliegue se realizará mediante el archivo Dockerfile, que será el encargado de construir y arrancar la aplicación en el servidor.

Después, se deben configurar las variables de entorno necesarias creando el archivo .env dentro del panel de configuración de Render. En este archivo se incluyen datos como la conexión a la base de datos, la configuración del correo electrónico, las claves de la aplicación y las credenciales de servicios externos.

Además del servicio web principal, es necesario crear otro servicio independiente para la base de datos. En este caso, se utiliza una base de datos compatible con el proyecto y debe configurarse con el mismo nombre y credenciales indicadas en el archivo .env.

Como Render, en su modalidad gratuita, no permite ejecutar comandos manualmente de forma sencilla, las migraciones de Laravel se realizaron mediante una ruta web creada en Laravel. A través de esta ruta se ejecutó el comando:

```
php artisan migrate
```

De esta forma, se pudo crear la estructura de la base de datos directamente desde el navegador. Por otro lado, para que el sistema de correos electrónicos y generación de PDFs funcione correctamente, fue necesario crear un servicio adicional de tipo Background Worker. Este servicio se encarga de ejecutar los procesos en segundo plano, como el envío de confirmaciones de reserva por correo electrónico.

Finalmente, para los recordatorios automáticos de reservas, se configuró un servicio de tipo Cron Job, encargado de ejecutar tareas programadas en determinados intervalos de tiempo.

Gracias a esta configuración, la aplicación queda desplegada en producción con su frontend, backend, base de datos, sistema de correos y tareas automáticas funcionando correctamente.

8.5. Problemas comunes y soluciones

Durante el desarrollo, instalación y despliegue de la aplicación surgieron diversos problemas técnicos relacionados principalmente con la configuración del entorno, el despliegue en producción y la integración entre Laravel y React. A continuación, se describen algunos de los problemas más importantes y sus soluciones.

Problemas con las migraciones en Render

La versión gratuita de Render no permite ejecutar comandos manuales fácilmente desde consola, lo que impedía lanzar las migraciones directamente mediante terminal.

Solución: Se creó temporalmente una ruta en Laravel capaz de ejecutar:

```
php artisan migrate
```

y posteriormente:

```
php artisan db:seed
```

Esto permitió crear y poblar la base de datos directamente desde el navegador.

Problemas con el envío de correos electrónicos

Inicialmente, los correos de confirmación de reservas no se enviaban correctamente en producción.

Solución: Se descubrió que Laravel Queue necesitaba ejecutarse mediante un servicio independiente. Para solucionarlo, se configuró un servicio **Background Worker** en Render encargado de ejecutar:

```
php artisan queue:work
```

Gracias a ello, el sistema de correos y generación de PDFs comenzó a funcionar correctamente.

Problemas con las imágenes almacenadas

En el primer despliegue, las imágenes subidas por los usuarios dejaban de mostrarse correctamente.

Solución: Fue necesario recrear el enlace simbólico de Laravel mediante:

```
php artisan storage:link
```

y verificar que las carpetas de almacenamiento estuviesen correctamente configuradas.

Problemas de autenticación entre Laravel y React

Durante el desarrollo surgieron errores relacionados con autenticación y sesiones, especialmente al proteger rutas privadas desde React.

Solución

Se utilizó Laravel Sanctum junto con Context API para mantener la sesión autenticada y controlar correctamente el acceso según el rol del usuario.

Problemas de compatibilidad en producción

Laravel y React requerían un entorno configurado específicamente para funcionar conjuntamente en Render.

Solución

Se creó un archivo **Dockerfile** personalizado que automatiza la instalación de dependencias, compilación del frontend y arranque de la aplicación en producción.

9. Control de versiones y GitHub

9.1. URL del repositorio

<https://github.com/islena17/EasySpa>

9.2. Estructura de ramas

Para el desarrollo del proyecto se ha utilizado un sistema de control de versiones basado en Git y organizado mediante distintas ramas, permitiendo separar correctamente el desarrollo de nuevas funcionalidades, correcciones y la versión estable de la aplicación.

La estructura principal del repositorio se divide en dos ramas principales: main y develop.

Rama main

La rama main contiene la versión estable y finalizada de la aplicación. Esta rama representa el estado del proyecto listo para producción y desplegado en el servidor. En ella únicamente se integran funcionalidades previamente comprobadas y validadas, evitando así errores en la versión pública de la aplicación.

Rama develop

La rama develop se utiliza como rama principal de desarrollo. Todas las nuevas funcionalidades y correcciones se integran primero en esta rama antes de pasar definitivamente a main. De esta forma, develop actúa como entorno intermedio donde se prueban los cambios antes de ser desplegados en producción. A partir de esta rama se crean diferentes tipos de ramas secundarias.

Ramas de funcionalidades (feature/...)

Las ramas feature se utilizaron para desarrollar funcionalidades concretas de la aplicación de forma aislada, permitiendo trabajar en nuevas características sin afectar al resto del sistema. Algunas de las ramas utilizadas durante el desarrollo fueron:

feature/AdminDashboard

feature/AdminProfile

feature/ApiGoogleCalendar

feature/CorregirSchedule

feature/ReservationReminder

feature/ScheduleWM

feature/WebMasterCalendar

feature/WebMasterDashboard

feature/clientProfile

feature/home-site

feature/mail

Cada una de estas ramas se centraba en una funcionalidad específica. Por ejemplo:

- *feature/ApiGoogleCalendar* incorporó la integración con Google Calendar.
- *feature/ReservationReminder* añadió el sistema automático de recordatorios.
- *feature/mail* implementó el sistema de envío de correos electrónicos.
- *feature/WebMasterDashboard* desarrolló el panel principal del webmaster.
- *feature/public-site* se centró en la parte pública de la página web.

Una vez finalizado el desarrollo de cada funcionalidad, la rama correspondiente se fusionaba (merge) con develop para realizar pruebas conjuntas antes de integrarse finalmente en main.

También es importante destacar que, al inicio del desarrollo, debido a la falta de experiencia previa en proyectos de gran tamaño y trabajo organizado con múltiples ramas en Git, en algunas ocasiones las ramas de funcionalidades se fusionaban accidentalmente con otras ramas distintas de develop. Esto provocó que ciertos cambios quedaran mezclados antes de tiempo, dificultando temporalmente la organización del repositorio y el control de versiones.

Sin embargo, conforme avanzó el desarrollo del proyecto, se adoptó una estructura de trabajo más organizada basada en una rama principal de desarrollo (develop). A partir de ese momento, cada nueva funcionalidad o corrección se desarrollaba en una rama independiente y posteriormente se fusionaba correctamente con develop antes de integrarse finalmente en main.

Esta metodología permitió mantener una estructura mucho más limpia, organizada y segura durante las fases finales del desarrollo del proyecto.

Ramas de corrección (Corregir...)

Además de las ramas de funcionalidades, también se utilizaron ramas destinadas exclusivamente a corregir errores o mejorar partes ya existentes de la aplicación.

CorreccionesMinimasFront

CorregirPaginación

CorregirShowSpaWM

Estas ramas permitieron solucionar errores concretos detectados durante las pruebas, mejorar la experiencia de usuario y optimizar diferentes apartados de la interfaz y lógica de la aplicación.

9.3. Convenciones de commits

Durante el desarrollo del proyecto se siguió una nomenclatura descriptiva para los commits realizados en Git, permitiendo identificar fácilmente los cambios introducidos en cada actualización del repositorio.

Los mensajes de commit se organizaron principalmente según el tipo de modificación realizada:

- Nuevas funcionalidades
- Corrección de errores
- Mejoras visuales
- Refactorización de código
- Configuración y despliegue

Algunos ejemplos utilizados durante el desarrollo fueron:

```
18e242b añadido algunos iconos bootstrap al diseño, y añadido showCategory y showReservation
e0403e3 mejoras en diseño responsive en el layout
e24ccdf correccion de errores en CreateUser, y hago showUser y su hooker
```

```
df9c924 (CorregirPaginación) corregir errores en paginación
596434e (feature/CorregirSchedule) correcciones: corrección en schedule de emple
ados y corrección en paginacion de reservas
478970f (feature/ReservationReminder) cambios en migracion
9fdbccf implemento mail de Recordatorio, vista, y cambios en modelo de Reserva,
también migracion tabla reserva
```

9.4. Cómo revisar el historial de versiones

El historial de versiones del proyecto se gestionó mediante Git, permitiendo registrar todos los cambios realizados durante el desarrollo de la aplicación. Para consultar el historial de commits y versiones del proyecto se puede utilizar el siguiente comando desde la terminal:

git log

Este comando muestra una lista completa de commits realizados en el repositorio, incluyendo:

- Identificador del commit
- Autor
- Fecha

- Mensaje descriptivo del cambio realizado

También es posible obtener una versión resumida del historial mediante:

git log --oneline

Gracias a Git, fue posible mantener un seguimiento organizado de la evolución del proyecto, facilitando la corrección de errores, la incorporación de nuevas funcionalidades y la recuperación de versiones anteriores en caso necesario. Además, el uso de ramas permitió desarrollar funcionalidades de forma independiente antes de integrarlas en la versión principal del sistema.

9.5 Capturas de hitos importantes del proyecto

Comienzos del proyecto (parte de backend)

```
MINGW64~/C:/Users/islen/Documents/tfg/Aplicacion-Web-de-Reservas-de-Spa/easy_spa
Author: Islena <islenapolo17@gmail.com>
Date: Sun Apr 5 15:44:05 2026 +0200

añado el service de availability para poder ver la disponibilidad de los servicios (me falta mejorarlo un poco)

commit 838f7003afab51a3d2a8a4b0c3c67dc1404072b6
Author: Islena <islenapolo17@gmail.com>
Date: Thu Apr 2 12:28:11 2026 +0200

añado los controllers api de cliente y publicos y cambio las rutas

commit dfe0375f6f1c5d5a4917f53d7916035ab681fa5f
Author: Islena <islenapolo17@gmail.com>
Date: Sun Mar 29 20:51:04 2026 +0200

añado los controllers de los empleados y cambio en rutas api

commit 1d1d71812c9520788664d6e0d66c443e4cd90d50
Author: Islena <islenapolo17@gmail.com>
Date: Sun Mar 29 19:49:25 2026 +0200

añado los demas controlers de admin

commit e59bc11796be3897bebb539f890d616ebe9d7937
Author: Islena <islenapolo17@gmail.com>
Date: Sun Mar 29 18:43:28 2026 +0200

corrijo error en spacontroller, añado carpeta api admin y clases employee y spaprofilecontroller

commit 7e6efae29d8be35009078c9b03cf072119b07b2d
Author: Islena <islenapolo17@gmail.com>
Date: Sun Mar 29 16:34:45 2026 +0200

controlles API (webmaster) y clases requests de Client, EmployeeBlock, EmployeeSchedule, SpaSchedule y Reservation

commit d9fe4035aa2d5bb4ff4ea67a7c2cf6bdd7ea100e
Author: Islena <islenapolo17@gmail.com>
Date: Sun Mar 29 14:36:59 2026 +0200

controllers y requests de service, employee, serviceCategory y Spa

commit 4aabb3eb21e1fe04855e2c02581e06e4984efe95
Author: Islena <islenapolo17@gmail.com>
Date: Mon Mar 23 10:01:28 2026 +0100

hago: Login/RegisterRequest, añado sanctum, hago controllers de auth, modifico config con sanctum, modifico rutas api con middleware (Webmaster)

commit e56fb36f1a1f99834d733f087f1a0b227102aa8a
Author: Islena <islenapolo17@gmail.com>
Date: Sat Mar 21 22:00:44 2026 +0100

hago el seeder de role y de user para añadir el webmaster y comienzo el controlador api de SpaController (Webmaster)

commit 351c82876f5442ecf71eb5c0fde73fcb460ac71d
Author: Islena <islenapolo17@gmail.com>
Date: Sat Mar 21 21:10:18 2026 +0100

creación de los modelos y sus migraciones: Spa, user, employee, client, service, category_service, employee_schedule, employee_blocks, reservation

commit 86376827039181d40596ade49dea2d7b50c2dbb3
Author: Islena <islenapolo17@gmail.com>
Date: Mon Mar 16 22:32:41 2026 +0100

instalacion del composer, react y laravel + vite

commit 59fbf41fd5b067d429d8f493b50596cfa8b47bd0
Author: Islena <78654052+islena17@users.noreply.github.com>
Date: Mon Mar 16 22:02:08 2026 +0100

Initial commit
(END)
```

Comienzo de creación del apartado Front-end (React) y algunos ajustes al back-end

```
MINGW64/C:/Users/islens/Documents/ttg/Aplicacion-Web-de-Reservas-de-Spa/easy_spa

commit d435dd41be0682aefb729f6cb75b44ef22fa4b52
Author: Islens <islensapolio17@gmail.com>
Date: Thu Apr 30 17:29:57 2026 +0200

    modifiko SpaController (show) para que se vean los clientes que tienen reservas en cada spa, hago el hook useClientForm, la vista edit y actualizo spaShow para que salgan ta

commit 14d4327f20517c8d903697497a63034bf7431152
Author: Islens <islensapolio17@gmail.com>
Date: Thu Apr 30 16:25:50 2026 +0200

    añadido editEmployee

commit c212ed93f4dee130d64c61c922d48cb9645c0917
Author: Islens <islensapolio17@gmail.com>
Date: Thu Apr 30 13:59:42 2026 +0200

    arreglo error en service controller y añadido la zona de empleado y hago hook y vista createEmployee

commit 45f1e3658739cca7bc4cd996c437a50646ece0c4
Author: Islens <islensapolio17@gmail.com>
Date: Thu Apr 30 12:29:15 2026 +0200

    añadido edit y updateService y modifiko el controller API porque llamaba mal a su relacion con Category

commit 6a700ca08c6d613235cefec1225bfff1c4b2970ff
Author: Islens <islensapolio17@gmail.com>
Date: Wed Apr 29 22:51:10 2026 +0200

    añadido creacion de servicio y su hook y empiezo la vista de edit y el hook (sin terminar)

commit 2380d28e9cf6f2dbe7712cf2f0e2ec6083392082
Author: Islens <islensapolio17@gmail.com>
Date: Wed Apr 29 21:58:00 2026 +0200

    añadido funcion de eliminar reserva

commit 63fd319b6942a710db7a4bbe8b14343bcea24ec0
Author: Islens <islensapolio17@gmail.com>
Date: Wed Apr 29 21:36:17 2026 +0200

    añadido boton de ver a index spa, añadido updateReservation en el hook de useReservationForm y añadido la vista edit

commit 24ab68b2c417f5e908bd7c85a912640257fdd126
Author: Islens <islensapolio17@gmail.com>
Date: Tue Apr 28 11:01:34 2026 +0200

    cambios en el hook de crear reserva porque no calculaba bien la hora ni el precio, tambien cambio en formulario para poder crear cliente nuevo

commit 76df2c0fb91b43f288b48e669625fa808f76fda3
Author: Islens <islensapolio17@gmail.com>
Date: Tue Apr 28 10:12:03 2026 +0200

    cambios en controller API Services y Reservation y sus requests (para hacer transaccion de crear cliente al crear reserva). Implemento employee, service y category seeder (D
dos)

commit c253bb3b61011f791ca3a54fc0d56af6b95fa332
Author: Islens <islensapolio17@gmail.com>
Date: Mon Apr 27 22:40:53 2026 +0200

    cambios en formularios edit, create spa. Añado show spa y empiezo las vistas de crear y edit reservas de spa. Separo la logica de los hooks en carpeta separada

commit bede9302d1fb7e6fe9013b2e2b85c1d5dfc59144
Author: Islens <islensapolio17@gmail.com>
Date: Sat Apr 11 15:32:11 2026 +0200

    cambio en authcontroller, cambios en rutas web y api y empiezo a hacer la parte de react

commit d115b322c0eae146d0632309cf4784760eea4a96
```

Modificación en código para hacer los componentes más reutilizables:

```
commit 0cf166efc3d624668570efc1b6ca2b2e5cb70dc4
Author: Islens <islensapolio17@gmail.com>
Date: Mon May 4 15:38:08 2026 +0200

    separo formulario de client

commit 32bf72e1f450034be85b924d3adc83dbe9c73578
Author: Islens <islensapolio17@gmail.com>
Date: Mon May 4 15:28:03 2026 +0200

    separo el formulario reservation

commit c2b2ae1c6e0f87925e73d33813d5840d3886637a
Author: Islens <islensapolio17@gmail.com>
Date: Mon May 4 15:17:29 2026 +0200

    separo formularios de service y category y modifiko el metodo destroy de category para que no pueda eliminar categorias con servicios asociados

commit dc9a8bea646ccd2ea5f32146e4055bc8af4d9479
Author: Islens <islensapolio17@gmail.com>
Date: Sun May 3 12:45:23 2026 +0200

    empiezo a separar formularios para hacer el codigo reutilizable
```

Creación de los controlers y hooks del Calendario de reservas

```
commit f46e3ea943739a7888a8b69682261183901f45e7
Author: Islena <islenapolo17@gmail.com>
Date: Tue May 5 23:21:49 2026 +0200

    cambios en layout, cambios en formulario employee para añadir el color de la tabla del calendario

commit 62259967d0e6967cb887d216447568d8097cf445
Author: Islena <islenapolo17@gmail.com>
Date: Tue May 5 23:12:05 2026 +0200

    creación de calendario y modificacion de controller (useCalendar, CalendarReservation)

commit 5efe21deb5453664cd587a87dcdf68b354b65795
Author: Islena <islenapolo17@gmail.com>
Date: Tue May 5 22:17:03 2026 +0200

    creo calendarcontroller e instalo fullcalendar
```

Implementación de recordatorios (emails)

```
commit 478970fbd180e67894c60ba599bbb4c7a1ce5970 (feature/ReservationReminder)
Author: Islena <islenapolo17@gmail.com>
Date: Thu May 14 11:58:46 2026 +0200

    cambios en migracion

commit 9fdbccf14711338778ea7eaddb5f32c162e75c31
Author: Islena <islenapolo17@gmail.com>
Date: Wed May 13 23:25:29 2026 +0200

    implemento mail de Recordatorio, vista, y cambios en modelo de Reserva, también migracion tabla reserva
```

Integración de API Google Calendar:

```
commit 6346223349ed4b996fea4c091bec6cd64bb35215
Author: Islena <islenapolo17@gmail.com>
Date: Sat May 16 23:25:35 2026 +0200

    integro api google calendar para que cuando se cree reserva-> se genere un boton con link a googlecalendar

commit 6ce1e2026355f9f36b2003de4f968bbea9d715c0 (feature/ScheduleMM)
```

Capturas de preparación para el despliegue:

```
commit bb63460430831ff4a62876b1dff5787bf9327f57
Author: Islena <78654052+islena17@users.noreply.github.com>
Date: Sun May 17 21:20:02 2026 +0200
```

Update Dockerfile

```
commit d4e84297a461e6e79cf2f6be2403b1ae10ba1095
Author: Islena <78654052+islena17@users.noreply.github.com>
Date: Sun May 17 20:06:20 2026 +0200
```

añadir Dockerfile para despliegue en Render

```
commit bc939dfe050b3cb77b6d3e6ec164c25dd5679fda
Author: Islena <78654052+islena17@users.noreply.github.com>
Date: Mon May 18 17:14:48 2026 +0200
```

actualizar baseURL de Axios para producción

10. Pruebas y validación

Durante el desarrollo de la aplicación se han realizado diferentes pruebas con el objetivo de comprobar que las funcionalidades principales funcionan correctamente y que los distintos roles del sistema pueden utilizar la plataforma de forma adecuada.

Las pruebas se han llevado a cabo de forma manual, utilizando datos de ejemplo generados mediante seeders y comprobando el comportamiento de la aplicación tanto desde el front-end como desde el back-end. También se han revisado las respuestas de la API, la persistencia de los datos en la base de datos y la correcta navegación entre las distintas secciones.

10.1. Casos de prueba principales

Caso de prueba	Descripción	Resultado esperado
Registro de usuario	Comprobar que un nuevo cliente puede registrarse correctamente en la aplicación.	El usuario se crea en la base de datos y puede iniciar sesión.
Inicio de sesión	Verificar que un usuario registrado puede acceder con sus credenciales.	El sistema autentica al usuario y lo redirige a la zona correspondiente según su rol.

Caso de prueba	Descripción	Resultado esperado
Protección de rutas	Acceder a rutas privadas sin iniciar sesión.	El sistema impide el acceso y redirige al login.
Listado de spas	Consultar los spas disponibles desde la parte pública.	Se muestran correctamente los spas activos con su información e imagen.
Detalle de spa	Acceder a la información de un spa concreto.	Se muestran los datos del spa y sus servicios asociados.
Reserva de servicio	Realizar una reserva seleccionando fecha, hora y servicio.	La reserva se guarda correctamente y se actualiza la disponibilidad.
Validación de disponibilidad	Intentar reservar una franja horaria ya ocupada.	El sistema impide crear reservas solapadas.
Gestión de servicios	Crear, editar o desactivar servicios desde el panel de administración.	Los cambios se guardan y se reflejan en la aplicación.
Gestión de empleados	Administrar empleados asociados a un spa.	Los empleados se crean, modifican o desactivan correctamente.
Gestión de horarios	Configurar horarios de empleados y del spa.	Los horarios se guardan y se tienen en cuenta al realizar reservas.

10.2. Pruebas de funcionalidad e integración

Las pruebas de funcionalidad se han centrado en comprobar que cada módulo de la aplicación cumple con el comportamiento esperado. Se han probado las operaciones principales de consulta, creación, edición y desactivación de datos en las entidades más importantes del sistema, como spas, servicios, empleados, horarios y reservas.

Entre las pruebas realizadas destacan:

- Comprobación de autenticación y mantenimiento de sesión.
- Validación de rutas protegidas según el rol del usuario.
- Comprobación de creación de reservas desde el formulario del cliente.
- Verificación de que una reserva genera el bloqueo correspondiente del empleado.
- Validación de que no se permiten reservas fuera del horario del spa o del empleado.
- Comprobación de que los servicios sin empleado permiten reservas por capacidad.
- Revisión del correcto funcionamiento de los paneles de cliente, administrador y webmaster.
- Comprobación de que los datos insertados mediante seeders se muestran correctamente en la aplicación.

10.3. Pruebas de accesibilidad

Además de las pruebas funcionales, se han tenido en cuenta aspectos básicos de accesibilidad y usabilidad para mejorar la experiencia de usuario. La aplicación se ha diseñado procurando que la navegación sea clara, que los formularios sean comprensibles y que los mensajes de error ayuden al usuario a corregir los datos introducidos.

Aspecto revisado	Validación realizada
Navegación clara	Las secciones principales son accesibles desde menús o botones visibles.
Formularios comprensibles	Los campos tienen etiquetas o indicaciones claras.
Mensajes de error	Se muestran avisos cuando los datos introducidos no son válidos.
Diseño responsive	Las vistas se adaptan a distintos tamaños de pantalla.
Botones identificables	Las acciones principales se muestran mediante botones visibles y coherentes.

Estas pruebas ayudan a garantizar que la aplicación sea más fácil de utilizar y que los usuarios puedan realizar las acciones principales sin dificultad, tanto desde ordenador como desde dispositivos con menor tamaño de pantalla.

11. Seguridad y buenas prácticas

11.1 Medidas de seguridad implementadas

Durante el desarrollo de la aplicación se implementaron diferentes medidas de seguridad con el objetivo de proteger tanto los datos de los usuarios como el correcto funcionamiento del sistema.

Autenticación y control de acceso

La aplicación utiliza Laravel Sanctum para gestionar la autenticación de usuarios y mantener sesiones seguras. Además, el sistema diferencia distintos roles de usuario, limitando el acceso a determinadas rutas y funcionalidades según los permisos correspondientes.

Protección de rutas

Tanto en el frontend como en el backend se implementó protección de rutas privadas para impedir el acceso no autorizado a paneles y funcionalidades restringidas.

Validación de formularios

Los formularios fueron validados tanto en el cliente como en el servidor:

- Validación frontend mediante React y expresiones regulares.
- Validación backend mediante Laravel.

Esto ayuda a prevenir errores, datos inválidos y posibles ataques derivados de entradas incorrectas.

Uso de variables de entorno

Las credenciales sensibles, como claves de APIs, configuración SMTP y datos de conexión a la base de datos, se almacenaron mediante variables de entorno en el archivo .env, evitando exponer información privada en el código fuente.

Gestión segura de contraseñas

Las contraseñas de los usuarios se almacenan cifradas utilizando el sistema de hashing integrado de Laravel.

Protección frente a acceso no autorizado

Se utilizaron middlewares de Laravel para verificar la autenticación y permisos de los usuarios antes de permitir el acceso a determinadas funcionalidades.

Control de sesiones

La aplicación controla el estado de autenticación mediante Context API y Laravel Sanctum, manteniendo sesiones persistentes y evitando accesos indebidos.

11.2 Buenas prácticas adoptadas

Durante el desarrollo del proyecto se siguieron diferentes buenas prácticas de programación y organización con el objetivo de mantener un código más limpio, escalable y mantenible.

Organización modular del proyecto

La aplicación se dividió en módulos y funcionalidades independientes, separando correctamente frontend, backend, hooks, componentes y lógica de negocio..

Separación entre frontend y backend

Laravel se utilizó como API REST y React como frontend independiente, facilitando una arquitectura más clara y mantenible.

Uso de control de versiones

Se empleó Git para gestionar el desarrollo del proyecto mediante ramas organizadas (main, develop y feature/...), permitiendo trabajar de forma más segura y estructurada.

Commits descriptivos

Los commits se realizaron utilizando mensajes descriptivos para facilitar el seguimiento de cambios y la evolución del proyecto.

Reutilización de componentes

Se desarrollaron componentes reutilizables en React para mantener una interfaz consistente y reducir código repetido.

Gestión centralizada del estado

Se utilizó Context API para gestionar el estado global de autenticación y usuario dentro de la aplicación.

Uso de colas y procesos en segundo plano

El envío de correos y recordatorios automáticos se implementó mediante queues y workers de Laravel, evitando bloquear la aplicación durante procesos pesados.

Despliegue mediante Docker

Para facilitar el despliegue en producción se utilizó un archivo Dockerfile que automatiza la instalación y configuración del entorno necesario para ejecutar la aplicación.

12. Mantenimiento y futuras mejoras

12.1. Cómo actualizar y mantener el proyecto

El proyecto ha sido desarrollado siguiendo una estructura modular que facilita su mantenimiento y futura ampliación. Gracias al uso de Laravel, React y TypeScript, es posible actualizar funcionalidades o corregir errores de forma organizada sin afectar al resto del sistema.

El mantenimiento de la aplicación se realiza principalmente mediante:

- Actualización de dependencias con Composer y NPM.
- Corrección de errores detectados durante el uso de la plataforma.
- Optimización del rendimiento tanto del frontend como del backend.
- Revisión de seguridad y actualización de librerías.
- Gestión del sistema de colas y tareas programadas.
- Mantenimiento de la base de datos y copias de seguridad.
- Control de versiones mediante Git para mantener un historial organizado de cambios.

Además, el uso de ramas de desarrollo permite implementar nuevas funcionalidades de forma segura antes de incorporarlas a la versión principal de producción.

12.2. Funcionalidades pendientes o ideas futuras

Aunque la aplicación cuenta actualmente con las funcionalidades principales necesarias para la gestión de reservas de spas, existen diversas mejoras y ampliaciones que podrían incorporarse en futuras versiones del proyecto.

Sistema de pagos online

Una de las mejoras más importantes sería la incorporación de una pasarela de pago online que permita a los clientes realizar pagos directamente desde la plataforma al efectuar una reserva.

Entre las posibles integraciones destacan:

- Stripe
- PayPal
- Redsys

Esto permitiría automatizar pagos, reservas y posibles cancelaciones o devoluciones.

Sistema de facturación

Se plantea también implementar un sistema de facturación para los spas, permitiendo generar automáticamente facturas relacionadas con reservas, servicios y pagos realizados por los clientes.

Este sistema podría incluir:

- Generación automática de facturas PDF

- Historial de facturación
- Gestión de impuestos
- Descarga de facturas por parte del cliente.

Integración con software hotelero y CRM

Como mejora futura, la aplicación podría integrarse con software utilizado habitualmente en hoteles y establecimientos turísticos, permitiendo sincronizar reservas y clientes entre distintos sistemas.

Algunos ejemplos serían:

- Ulyses CRM
- Hera CRM

Esta integración facilitaría que hoteles con spa pudiesen gestionar toda la información desde una única plataforma centralizada.

Sistema de inventario y productos

Otra posible mejora sería incorporar un sistema de gestión de productos utilizados en los spas.

Cada spa podría disponer de su propio inventario para controlar:

- Productos cosméticos
- Aceites y tratamientos
- Materiales utilizados en servicios
- Stock disponible
- Alertas de reposición

Esto permitiría una gestión más completa del negocio dentro de la misma plataforma.

Aplicación móvil

También se podría desarrollar una aplicación móvil para Android e iOS que permita a los clientes gestionar reservas de forma más cómoda desde dispositivos móviles.

13. Conclusión

13.1. Valoración del proyecto

El desarrollo de este proyecto ha permitido crear una aplicación web completa orientada a la gestión de reservas de spas, integrando funcionalidades tanto para clientes como para empresas y administradores.

A lo largo del desarrollo se han implementado múltiples tecnologías actuales del entorno web, combinando Laravel como backend y React con TypeScript como frontend, además

de herramientas complementarias como FullCalendar, Docker, MySQL y sistemas de colas para correos y tareas automáticas.

El proyecto no solo ha cumplido los objetivos principales planteados inicialmente, sino que además ha evolucionado incorporando nuevas funcionalidades conforme avanzaba el desarrollo, como la integración con Google Calendar, el sistema de recordatorios automáticos o la generación de PDFs.

Además, el resultado final ha permitido obtener una plataforma funcional, escalable y preparada para seguir creciendo en futuras versiones.

13.2. Aprendizajes obtenidos

La realización de este TFG ha supuesto un aprendizaje muy importante tanto a nivel técnico como organizativo.

A nivel técnico, he adquirido conocimientos relacionados con:

- Desarrollo full stack con Laravel y React.
- Creación y consumo de APIs REST.
- Gestión de autenticación y roles.
- Uso de TypeScript y hooks personalizados.
- Gestión de estados mediante Context API.
- Integración de servicios externos como Google Calendar.
- Uso de FullCalendar para gestión visual de reservas.
- Despliegue de aplicaciones mediante Docker y Render.
- Configuración de colas, workers y cron jobs.
- Gestión de bases de datos relacionales con MySQL.

También he aprendido buenas prácticas relacionadas con la organización de proyectos grandes, el uso de Git y la gestión de ramas durante el desarrollo.

Además, uno de los aprendizajes más importantes ha sido la capacidad de resolver problemas reales surgidos durante el desarrollo y despliegue de la aplicación, investigando soluciones y adaptándose a nuevas tecnologías y herramientas.

En conjunto, este proyecto ha permitido consolidar muchos de los conocimientos que he adquirido durante el ciclo formativo y aplicarlos en una aplicación real, completa y funcional.

14. Anexo

14.1. Código relevante comentado

En este apartado se incluyen fragmentos de código relevantes para el funcionamiento principal de la aplicación. Se ha seleccionado especialmente el CRUD de reservas, tanto en la parte del backend desarrollada con Laravel como en la parte del frontend desarrollada con TypeScript y React.

También se incluye el servicio encargado de calcular la disponibilidad de los empleados y del spa, ya que es una parte fundamental del sistema. Este servicio permite comprobar qué trabajadores están disponibles en una fecha y franja horaria concreta, evitando solapamientos y garantizando que las reservas se realicen correctamente.

14.1.1. Creación de reservas con Laravel

- Metodo Store de ReservationController

```
/* Este método utiliza una transacción para asegurar que todas
las operaciones relacionadas con la reserva se realizan correctamente.
* Si ocurre algún error durante el proceso, se revierte la
operación.
*/
```

```
public function store(ReservationRequest $request)
{
    $reservation = DB::transaction(function () use ($request) {
        // Se obtienen únicamente los datos validados por
ReservationRequest.
        $data = $request->validated();

        // Se busca el servicio asociado a la reserva.
        $service = Service::findOrFail($data['service_id']);

        /*Se comprueba que el número de personas indicado no supere la
capacidad máxima permitida para el servicio seleccionado. */

        if (
            Reservation::exceedsServiceCapacity(
                $service,
                $data['number_of_people']
            )
        ) {
            throw ValidationException::withMessages([
                'number_of_people' => [
                    'La capacidad máxima de este servicio es de '
. $service->capacity . ' personas.',
                ],
            ]);
        }
    });
}
```



```

/*Se guarda el precio base del servicio en el momento de realizar la
reserva. Esto evita que una futura modificación del precio del
servicio altere reservas ya existentes. */

$data['service_price'] = $service->price;

/*Se calcula el precio final de la reserva en función del servicio
seleccionado y del número de personas. */
$data['final_price'] = Reservation::calculateTotalPrice(
    $service,
    $data['number_of_people']
);

/*Si la reserva no está asociada a un cliente existente,
pero se reciben los datos de un nuevo cliente, se crea dicho cliente
antes de registrar la reserva.*/
if (empty($data['client_id']) && isset($data['client'])) {
    $client = Client::create([
        'name' => $data['client']['name'],
        'surname' => $data['client']['surname'],
        'email' => $data['client']['email'] ?? null,
        'telephone' => $data['client']['telephone'] ??
null,
    ]);

    $data['client_id'] = $client->id;
}

// Se elimina el array auxiliar de cliente para evitar insertar datos
que no pertenecen directamente a la tabla reservas.
unset($data['client']);

// Finalmente, se crea la reserva en la base de datos.
return Reservation::create($data);
});

// Se devuelve la reserva creada junto con sus relaciones principales.
return response()->json([
    'message' => 'Reserva creada correctamente',
    'data' => $reservation->load([
        'client',
        'spa',
        'service',
        'employee',
    ]),
], 201);

```

14.1.2. Creación de reservas en React y TypeScript

- Función createReservation

Este fragmento muestra la creación de una reserva desde el frontend. Si el administrador introduce un cliente nuevo, primero se crea el cliente y posteriormente se envía la reserva al backend.

```
const createReservation = async (e: FormEvent) => {
  e.preventDefault();

  setLoading(true);
  setErrors({});

  try {
    let clientId = form.client_id;

    if (showClientForm) {
      const clientRes = await api.post('/api/admin/clients', {
        name: clientForm.name,
        surname: clientForm.surname,
        email: clientForm.email || null,
        telephone: clientForm.telephone || null,
      });

      const newClient = clientRes.data.data ?? clientRes.data;
      clientId = String(newClient.id);
    }

    await api.post('/api/admin/reservations', buildPayload(clientId));

    navigate('/admin/reservations');
  } catch (error: any) {
    handleRequestError(error, 'Ha ocurrido un error al crear la reserva.');
```

14.1.3. Cálculo de disponibilidad en Laravel

-Método buildSlotsForEmployee de AvailabilityService

```
/*Obtiene los huecos disponibles para realizar una reserva. Este
método comprueba la disponibilidad de un servicio en un spa concreto,
teniendo en cuenta la fecha seleccionada, el horario del spa, el
horario de los empleados y las reservas ya existentes. */

public function getAvailableSlots(
    int $spaId,
    int $serviceId,
    string $date,
    ?int $employeeId = null,
    int $interval = 30
): array {
    /*Se obtiene el servicio solicitado, comprobando que pertenece al
    spa indicado y que se encuentra activo. */
    $service = Service::where('spa_id', $spaId)
        ->where('is_active', true)
        ->findOrFail($serviceId);

    /* Se obtiene el día de la semana correspondiente a la fecha
    seleccionada. Esto permite buscar el horario general del spa para ese
    día.*/
    $dayOfWeek = Carbon::parse($date)->dayOfWeek;

    /*Se consulta el horario del spa para el día seleccionado.Si el
    spa no trabaja ese día, no se devuelve ningún hueco disponible.
    */
    $spaSchedule = SpaSchedule::where('spa_id', $spaId)
        ->where('day_of_week', $dayOfWeek)
        ->where('is_working', true)
        ->first();

    if (!$spaSchedule) {
        return [];
    }

    /*Se prepara la consulta de empleados pertenecientes al spa. Si el
    servicio requiere empleado y se ha seleccionado uno concreto, se
    filtra únicamente por ese empleado.*/
    $employeesQuery = Employee::where('spa_id', $spaId);

    if ($service->requires_employee) {
        if ($employeeId) {
            $employeesQuery->where('id', $employeeId);
        }
    } else {
        /*Si el servicio no requiere empleado, se calculan los huecos
        disponibles únicamente con el horario general del spa.*/
        return $this->getSlotsWithoutEmployee(
            $spaId,
            $service,
            $date,
            $spaSchedule->start_time,
```

```

        $spaSchedule->end_time,
        $interval
    );
}

$employees = $employeesQuery->get();

$slots = [];

/*Se recorren los empleados disponibles para comprobar su horario
individual y generar sus posibles franjas de reserva. */
foreach ($employees as $employee) {
    /*Se obtiene el horario del empleado para la fecha
    seleccionada. Si el empleado no trabaja ese día, se pasa al
    siguiente.*/
    $employeeSchedule = EmployeeSchedule::where('employee_id',
    $employee->id)
        ->where('date', $date)
        ->where('is_working', true)
        ->first();

    if (!$employeeSchedule) {
        continue;
    }

    /*Se calcula el intervalo real de disponibilidad combinando el
    horario del spa con el horario del empleado. La hora de inicio será la
    más tardía entre ambas y la hora de fin será la más temprana. */
    $startBoundary = $this->maxTime(
        $spaSchedule->start_time,
        $employeeSchedule->start_time
    );

    $endBoundary = $this->minTime(
        $spaSchedule->end_time,
        $employeeSchedule->end_time
    );

    /*Se generan los huecos disponibles para este empleado,
    teniendo en cuenta la duración del servicio y el intervalo indicado.*/
    $employeeSlots = $this->buildSlotsForEmployee(
        $employee->id,
        $date,
        $service->length_minutes,
        $startBoundary,
        $endBoundary,
        $interval
    );

    /*Se añaden los huecos generados al listado final, incluyendo
    el empleado asociado a cada franja horaria. */
    foreach ($employeeSlots as $slot) {
        $slots[] = [
            'employee_id' => $employee->id,
            'start_time' => $slot['start_time'],
            'end_time' => $slot['end_time'],
        ];
    }
}

```

```

    }
}

/*Se ordenan los huecos disponibles por hora de inicio para
mostrarlos de forma clara en el frontend.*/
usort($slots, function ($a, $b) {
    return strcmp($a['start_time'], $b['start_time']);
});

return $slots;
}

```

-Método buildSlotsForEmployee de AvailabilityService:

```

/* A partir de la hora de inicio, la hora de fin y la duración del
servicio, construye posibles huecos de reserva y comprueba que no se
solapen con reservas existentes ni con periodos bloqueados del
empleado. */
private function buildSlotsForEmployee(
    int $employeeId,
    string $date,
    int $durationMinutes,
    string $startTime,
    string $endTime,
    int $interval
): array {
    $slots = [];

    // Se crean las fechas con Carbon para poder sumar minutos y
    comparar horarios.
    $start = Carbon::parse($date . '-' . $startTime);
    $end = Carbon::parse($date . '-' . $endTime);

    // Se generan huecos mientras el servicio pueda finalizar dentro
    del horario disponible.
    while ($start->copy()->addMinutes($durationMinutes) <= $end) {
        // Se calcula la hora de finalización del hueco actual.
        $slotEnd = $start->copy()->addMinutes($durationMinutes);

        // Solo se añade el hueco si no existe una reserva o bloqueo
        en esa franja.
        if (
            !$this->hasOverlappingReservation($employeeId, $date,
            $start, $slotEnd) &&
            !$this->hasBlockingPeriod($employeeId, $date, $start,
            $slotEnd)
        ) {
            $slots[] = [
                'start_time' => $start->format('H:i'),
                'end_time' => $slotEnd->format('H:i'),
            ];
        }

        // Se avanza al siguiente posible hueco según el intervalo
        indicado.
        $start->addMinutes($interval);
    }
}

```

```

    }

    return $slots;
}

```

-Metodo hasOverlappingReservation de AvailabilityService

```

/**
 * Comprueba si un empleado ya tiene una reserva en la franja indicada.
 * Las reservas canceladas no bloquean disponibilidad.
 */
private function hasOverlappingReservation(
    int $employeeId,
    string $date,
    Carbon $start,
    Carbon $end
): bool {
    return Reservation::where('employee_id', $employeeId)
        ->where('reservation_date', $date)

        /**
         * Las reservas canceladas no bloquean disponibilidad,
         * por lo que no se tienen en cuenta en la comprobación.
         */
        ->where('status', '!=', 'cancelled')

        /**
         * Se comprueba el solapamiento de horarios.
         *
         * Existe solapamiento cuando una reserva empieza antes de que
termine
         * el nuevo hueco y termina después de que empiece dicho
hueco.
         */
        ->where(function ($query) use ($start, $end) {
            $query
                ->where('start_time', '<', $end->format('H:i:s'))
                ->where('end_time', '>', $start->format('H:i:s'));
        })
        ->exists();
}

```

14.1.4 Consulta de disponibilidad en React

-useEffect para obtener huecos disponibles

```
// Cada vez que cambia el servicio, la fecha, el empleado o la
reserva,
// se consulta al backend la disponibilidad actualizada.
useEffect(() => {
  const fetchAvailableSlots = async () => {
    // Si no se ha seleccionado servicio o fecha, no se buscan
    horarios.
    if (!form.service_id || !form.reservation_date) {
      setAvailableSlots([]);
      return;
    }

    try {
      setLoadingSlots(true);

      // Petición al backend para obtener los huecos disponibles.
      const res = await api.get('/api/admin/availability', {
        params: {
          service_id: form.service_id,
          date: form.reservation_date,
          employee_id: form.employee_id || undefined,
          exclude_reservation_id: reservationId,
        },
      });

      // Se guarda la lista de horarios disponibles recibida.
      const slots = res.data.data ?? res.data ?? [];
      setAvailableSlots(Array.isArray(slots) ? slots : []);
    } catch (error) {
      console.error(error);
      setAvailableSlots([]);
    } finally {
      setLoadingSlots(false);
    }
  };

  fetchAvailableSlots();
}, [form.service_id, form.reservation_date, form.employee_id,
reservationId]);
```

-selectSlot para seleccionar un hueco

```
// Al seleccionar un hueco disponible, se actualizan en el formulario
la hora de inicio, la hora de fin y, si existe, el empleado asignado.
const selectSlot = (slot: AvailableSlot) => {
  setForm((prev) => ({
    ...prev,
    start_time: slot.start_time,
    end_time: slot.end_time,
    employee_id: slot.employee_id ? String(slot.employee_id) :
prev.employee_id,
  }));
};
```

14.2. Diagramas adicionales:

Diagrama ER:

